

Equation Guide for the Sentinel Report

Md. Shifat Hasan, Roll 2107067 | CSE 4102

Each section matches one equation of Chapter III (numbers in brackets are the report's equation numbers). For every equation: what it does, what each symbol means, why it has that shape, a numeric example, the exact place in the code, and a short script for the viva. All formulas were re-checked against the source files; four statements in the first draft were corrected (the IK angle, the Cook–Torrance form, the normal-map scale and the wind formula) and the report now matches the code.

G1. Noise deformer (3.1): turning a sphere into a rock

$$\mathbf{p}' = \mathbf{p} + A \nu(f \mathbf{p}) \hat{n}, \quad \|\mathbf{p}' - \mathbf{p}\| \leq A.$$

Meaning. Every vertex $\mathbf{p}$ of a mesh is pushed along its surface direction $\hat{n}$ by a smooth random amount.

Symbols: A amplitude in metres (largest possible push); $\nu \in [-1, 1]$ a 3-D fractal noise value; f frequency, in cycles per metre (higher f gives smaller bumps, because the noise is sampled at $f\mathbf{p}$); $\hat{n}$ unit normal of the original surface. **Why the bound holds:** $|\nu| \leq 1$ and $\|\hat{n}\| = 1$, so the push is at most A ; this is what makes object size ranges testable. Neighbouring vertices get nearly equal ν (noise is smooth), so the surface stays connected and lumpy, not spiky.

Worked example (numbers checked by script)

$A = 0.3$ m and $\nu = 0.5$ at one vertex: the vertex moves $0.3 \times 0.5 = 0.15$ m outward. At $\nu = -1$ it moves 0.3 m inward.

Code: Engine/Procedural/Deform.odin, case Noise_Displace.

How to say it in 20 seconds

“A noise deformer moves each vertex along its normal by amplitude times smooth noise, so shapes get natural lumps while the maximum change is guaranteed to be the amplitude.”

G2. Two-bone inverse kinematics (3.2): placing a foot

$$\cos \alpha = \frac{a^2 + c^2 - b^2}{2ac}$$

Meaning. A leg has an upper bone (length a , thigh) and a lower bone (length b , shin). Given where the foot must be, we need the hip angle. The hip, knee and foot form a triangle with sides a , b and $c = |\text{target} - \text{hip}|$; the *law of cosines* gives any angle from the three sides. **Symbols:** α angle at the hip between the hip-to-target line and the thigh. The knee position then lies at distance $a \cos \alpha$ along the target line and $a \sin \alpha$ to the side. The interior knee angle θ uses the same law: $\cos \theta = (a^2 + b^2 - c^2)/2ab$. **Safety:** the code clamps c and the cosine to $[-1, 1]$, so an unreachable target gives a straight leg instead of a numerical error.

Worked example (numbers checked by script)

$a = b = 0.45$ m, foot target $c = 0.8$ m away: $\cos \alpha = 0.64/0.72 = 0.889$, so $\alpha = 27.3^\circ$ and the knee angle is 125.5° (bent but not folded). At $c = 0.9 = a + b$ the leg is straight ($\alpha = 0$).

Code: Game/Characters/Ik.odin (law of cosines in the header comment and body).

How to say it in 20 seconds

“Hip, knee and foot make a triangle whose sides I know, so the law of cosines gives the joint angle; no animation files are needed.”

G3. Fractal noise (3.3) and why textures tile

$$F(\mathbf{x}) = \frac{\sum_{i=0}^{n-1} 2^{-i} \nu(2^i \mathbf{x})}{\sum_{i=0}^{n-1} 2^{-i}}$$

Base noise ν . At each grid corner a random unit direction (one of 16, picked by a hash of the corner index) is stored. For a point inside a cell, ν takes the dot product of each corner direction with the vector from that corner to the point, then blends the four values with the smooth curve $s(t) = 6t^5 - 15t^4 + 10t^3$ (it has zero slope and zero curvature at 0 and 1, so no grid lines show), and multiplies by $\sqrt{2}$ so the result spans $[-1, 1]$. **Fractal sum.** Octave i has twice the frequency ($2^i \mathbf{x}$) and half the strength (2^{-i}): big hills, then boulders, then pebbles. The denominator is the total strength, so F stays in $[-1, 1]$. **Tiling.** The corner direction is chosen from the *cell index modulo the period P* (and P doubles each octave). Then $F(\mathbf{x} + P\mathbf{e}) = F(\mathbf{x})$ for whole-number shifts $\mathbf{e}$: the right edge of a texture equals its left edge, so repeated tiles have no seam. The test suite checks this exactly (shift $(1, -2)$ tiles; difference must be 0).

Worked example (numbers checked by script)

$n = 4$ octaves: weights $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}$ sum to 1.875. A value $\nu = +1$ in every octave gives $F = 1$; typical values mix and give about ± 0.4 . $s(0.5) = 0.5$ exactly, and $s(0.25) = 0.104$, so the blend is gentle near cell edges.

Code: Shaders/Include/Noise.glsl (gradient_noise, fbm).

How to say it in 20 seconds

“Fractal noise adds smooth random layers, each twice as detailed and half as strong; by wrapping the grid index it repeats exactly, so textures tile.”

Variants used: *domain warping* $F(\mathbf{x} + s[F_1(\mathbf{x}), F_2(\mathbf{x})])$ evaluates F at a point shifted by two other noises, which bends straight lattice patterns into swirls (still periodic, since every term is). *Ridged* noise uses $(1 - |\nu|)^2$: $|\nu|$ folds the noise at zero so zero crossings become sharp ridges (cracks, bark). *Cellular (Worley)* noise uses the distance to the nearest random point (leaves, pebbles).

G4. Normal map from a height map (3.4)

$$h_x \approx \frac{3(h_{NE} + h_{SE} - h_{NW} - h_{SW}) + 10(h_E - h_W)}{32 \Delta N}, \quad \hat{n} = \frac{(-k h_x, -k h_y, 1)}{\|(-k h_x, -k h_y, 1)\|}$$

Meaning. A recipe provides a height h per texel. Lighting only needs the surface *direction*, which tilts away from the slope: a surface rising to the right ($h_x > 0$) tilts its normal to the left ($-h_x$). **Scharr filter:** the slope is a weighted difference of the right column (3, 10, 3) minus the left column of the 3×3 neighbourhood. Weights sum to 16 over a distance of two texels, hence the divisor 32Δ ; it treats diagonal and axis-aligned slopes equally, which the plain two-point difference does not. **Symbols:** Δ one texel (here $1/1024$ of the tile); N cells per tile (so a bumpy recipe with more cells is not made steeper); k the per-material bump strength; the vector is normalised to unit length. The last component 1 means a flat area gives $\hat{n} = (0, 0, 1)$.

Worked example (numbers checked by script)

A perfect ramp $h = 0.7x$ (with $\Delta = 1$): the numerator is $3 \cdot 2.8 + 10 \cdot 1.4 = 22.4$, divided by 32 gives 0.7, the true slope (verified by script). With $k = 1$: $\hat{n} = (-0.7, 0, 1)/1.22 = (-0.57, 0, 0.82)$, tilted 35° .

Cavity darkening. Where h is below the mean of its four neighbours (a negative Laplacian), the albedo is multiplied by $1 - 0.5c$ and the ambient occlusion by $1 - c$, where $c \in [0, 0.5]$ grows with the depth of the dent. **Triplanar weights:** $w_i = |n_i|^4 / \sum_j |n_j|^4$ (the code uses exponent 4): a surface facing $+y$ mostly uses the top-down projection. For $\mathbf{n} = (0.6, 0.8, 0)$ the weights are 0.24, 0.76, 0. **Code:**

Shaders/Include/BakeMain.glsl, Shaders/Include/Triplanar.glsl.

How to say it in 20 seconds

“The height map’s slope tells how the surface tilts; a Scharr filter measures it fairly in every direction, and the normal is minus the slope with a one up, normalised.”

G5. Cook–Torrance reflection (3.5)

$$f = \frac{(1 - F_l)(1 - F_v) c_d}{\pi} + D V F, \quad D = \frac{\alpha^2}{\pi((\mathbf{n} \cdot \mathbf{h})^2(\alpha^2 - 1) + 1)^2}, \quad F = F_0 + (1 - F_0)(1 - \mathbf{v} \cdot \mathbf{h})^5$$
$$V = \frac{0.5}{(\mathbf{n} \cdot \mathbf{l})\sqrt{(\mathbf{n} \cdot \mathbf{v})^2(1 - \alpha^2) + \alpha^2} + (\mathbf{n} \cdot \mathbf{v})\sqrt{(\mathbf{n} \cdot \mathbf{l})^2(1 - \alpha^2) + \alpha^2}}$$

Meaning. f is the fraction of light arriving from direction $\mathbf{l}$ that leaves toward the viewer $\mathbf{v}$; the lighting pass multiplies it by the light’s brightness and $\mathbf{n} \cdot \mathbf{l}$. The surface is imagined as many tiny mirrors (microfacets).

Vectors: $\mathbf{n}$ surface normal, $\mathbf{l}$ to the light, $\mathbf{v}$ to the eye, $\mathbf{h}$ halfway between $\mathbf{l}$ and $\mathbf{v}$ (only facets facing $\mathbf{h}$ reflect light from $\mathbf{l}$ to $\mathbf{v}$). **Specular part DVF .** D (GGX): the share of facets facing $\mathbf{h}$; rough surfaces have small peak, wide lobe. $\alpha = \text{roughness}^2$. $V = G/(4\mathbf{n} \cdot \mathbf{l} \mathbf{n} \cdot \mathbf{v})$: the geometry term G (facets blocking each other) with the denominator folded in (height-correlated Smith form). F (Schlick): every surface reflects more at glancing angles; F_0 is reflectance looking straight on (0.04 for plastic, stone, paint; the albedo colour for metals). **Diffuse part.** c_d/π is the Lambert term (π makes total reflected light equal the colour). It is scaled by $(1 - F_l)(1 - F_v)$ because light that mirror-reflected at the surface cannot also scatter inside; this keeps energy ≤ 1 . Metals have $c_d = 0$.

Worked example (numbers checked by script)

Roughness 0.5 gives $\alpha = 0.25$. Peak of D at $\mathbf{n} \cdot \mathbf{h} = 1$: $1/(\pi \cdot 0.0625) = 5.09$. Fresnel with $F_0 = 0.04$: looking at 60° gives $F = 0.07$; at 85° (grazing) $F = 0.65$, why wet roads shine at low angles.

Code: Shaders/Include/Brdf.glsl, microfacet_cook_torrance.

How to say it in 20 seconds

“Light reflects as a diffuse part plus a specular part from microscopic mirrors: GGX says how many face the eye, Smith says how many hide each other, Schlick says how reflective at this angle.”

G6. Fog by distance (haze)

$$C = T C_{\text{surface}} + (1 - T) C_{\text{haze}}, \quad T = e^{-kd}, \quad k = 3.5 \times 10^{-4} \text{ m}^{-1}$$

Meaning. Air absorbs and scatters light. T is the fraction of a surface’s light that survives d metres of air (Beer–Lambert law: equal thickness removes an equal *fraction*, so the decay is exponential); the missing part is replaced by the sky-coloured haze, a linear mix. **Symbols:** d distance to the camera; k density.

Worked example (numbers checked by script)

$d = 100 \text{ m}$: $T = 0.966$ (almost clear). $d = 1 \text{ km}$: $T = 0.705$. $d = 3 \text{ km}$: $T = 0.35$, so far hills are mostly sky colour, giving depth. The haze colour uses only the smooth atmosphere (never stars or the moon), which fixed the night streak bug.

Code: Shaders/DeferredBase.glsl (FOG_DENSITY_PER_METER).

How to say it in 20 seconds

“Each metre of air removes the same percentage of light, so transmittance is an exponential, and what is lost is replaced by sky colour.”

G7. ACES tone mapping (3.6)

$$C_{\text{out}} = \text{clamp}_{[0,1]} \left(\frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14} \right), \quad x = \text{exposure} \times \text{radiance}$$

Meaning. Scene light (HDR) can be 0.01 in shade or 40 at the sun, but a display shows 0 to 1. This curve (Narkowicz’s fit of the ACES film curve) compresses highlights smoothly (shoulder), lifts dark detail gently (toe) and has an S-shaped contrast like film. **Symbols:** x radiance scaled by exposure (1 by day, rising to 6 at night, like the eye adapting); applied to each colour channel. The numbers are fitted constants.

Worked example (numbers checked by script)

$x = 0.18$ (mid-grey) $\rightarrow 0.267$; $x = 1 \rightarrow 0.804$; $x = 4 \rightarrow 0.973$. Quadrupling brightness from 1 to 4 changes the output by only 0.17: highlights do not clip to white abruptly.

After the curve the image is graded, converted to sRGB for the display and dithered by about one 8-bit step to avoid banding. **Code:** Shaders/Include/Tonemap.glsl, Shaders/PostTonemap.glsl.

How to say it in 20 seconds

“A rational curve maps unlimited brightness into zero-to-one with a soft shoulder, the way film does.”

G8. Sun height (inline in Section 3.2.5)

$$\sin(\text{elev}) = \sin \varphi \sin \delta + \cos \varphi \cos \delta \cos h, \quad h = 15^\circ (\text{hours} - 12)$$

Meaning. Standard spherical astronomy. $\varphi = 35^\circ$ latitude, $\delta = 15^\circ$ the sun’s declination (a late-spring day), h the hour angle (the Earth turns 15° per hour; $h = 0$ at noon). The full direction vector uses the same terms with east and south components. Positive result: sun above the horizon.

Worked example (numbers checked by script)

Noon: $\sin = 0.1485 + 0.7913 = 0.9398$, elevation 70° . At 6:00 and 18:00: 8.5° (low, orange sun). Midnight: -40° , so the moon (opposite the sun) lights the scene.

Code: Game/Gameplay/TimeOfDay.odin.

G9. Cloud plane and wind (3.7)

$$\mathbf{P}_{xz} = \frac{H}{d_y} (d_x, d_z) + \mathbf{w} t$$

Meaning. Clouds are drawn as a flat noise layer at height H . For a view direction $\mathbf{d} = (d_x, d_y, d_z)$, a ray from the ground reaches height H after travelling H/d_y times the direction (similar triangles); that gives the horizontal position $(d_x, d_z)H/d_y$ where the noise is looked up. Adding $\mathbf{w}t$ ($\mathbf{w}$ wind velocity in m/s, t time) slides the pattern, so clouds drift. A faster detail layer ($\times 1.35$) and a cirrus layer at 7 km ($\times 1.8$ speed) make shapes change and give depth. Rays near the horizon ($d_y < 0.04$) are skipped, because H/d_y would blow up.

Worked example (numbers checked by script)

$\mathbf{d} = (0.8, 0.5, 0.33)$ and $H = 1500$: $H/d_y = 3000$, point (2400, 995) m. With $\mathbf{w} = (14, 5)$ m/s the pattern moved (840, 300) m after one minute.

Code: Shaders/Include/Sky.glsl (add_clouds, cloud_density); wind in Game/Sandbox/Sandbox.odin.

G10. Tree sway (cantilever wind)

$$\Delta_x = 2s(0.04 + 0.1g)\left(\frac{y}{9}\right)^2, \quad \Delta_z = 0.35\Delta_x, \quad g = \frac{1}{2} + \frac{1}{2}\sin(1.3t + 0.05x + 0.03z)$$

Meaning. The vertex shader shifts each tree vertex sideways. A real tree is like a beam fixed at the root: it barely moves at the bottom and most at the top, so the shift grows with the *square* of height y (clamped to 9 m). $g \in [0, 1]$ is a gust that travels across the land (position terms $0.05x + 0.03z$ delay it from tree to tree, so trees do not sway in lockstep). s is the per-mesh sway (0.6 for trunks, 1 for foliage; foliage above 1 also adds a small fast flutter). The factor 2 and offsets 0.04/0.1 are tuned amplitudes: calm sway 0.08 m, full gust 0.28 m at the top.

Worked example (numbers checked by script)

Foliage ($s = 1$), full gust $g = 1$: at $y = 9$ m, $\Delta_x = 2(0.14)(1) = 0.28$ m; at half height 4.5 m it is 0.07 m (a quarter, because of the square).

Code: Shaders/Geometry.glsl, wind_offset.

G11. Vehicle steering (bicycle model) and engine power

$$\dot{\psi} = \frac{v \tan \delta}{L}, \quad \dot{v} = a_0 \left(1 - 0.6 \frac{|v|}{v_{\max}}\right)$$

Steering. Treat the car as a bicycle with wheel base L (m), front wheel angle δ , speed v . The car turns on a circle of radius $R = L/\tan \delta$, so the yaw rate is $v/R = v \tan \delta/L$ (rad/s). Faster means turning faster; reversing flips the sign, as in a real car. (Tracked vehicles instead turn at a fixed rate and can pivot.) **Power fade.** Engine thrust is power divided by speed, so acceleration is high from rest and falls as speed rises. a_0 is the maximum acceleration, $v_{\max}$ the limit; at top speed acceleration is still $0.4a_0$ but the speed is clamped to the limit.

Worked example (numbers checked by script)

Jeep ($L = 3$, $\delta_{\max} = 0.55$ rad, $v_{\max} = 18$, $a_0 = 2.6$): at 10 m/s full lock gives $\dot{\psi} = 10 \tan 0.55/3 = 2.04$ rad/s. Solving the speed equation, $v(t) = 30(1 - e^{-0.0867t})$: 2.5 m/s after 1 s, and 18 m/s after 10.6 s (it was 24 m/s with 7 m/s^2 before the realism change).

Code: Engine/World/GroundVehicle.odin (yaw_rate_for, next_speed).

Short answers to likely questions

- **Why not use image textures?** The project rule forbids files; noise gives infinite variety from a seed and tiles exactly.
- **Why a quintic fade and not linear blending?** Linear blending shows grid lines (slope jumps at cell edges); $6t^5 - 15t^4 + 10t^3$ has zero first and second derivative there.
- **Why $\sqrt{2}$ in the noise?** Unit gradients bound the raw value by $\sqrt{2}/2 \approx 0.707$; multiplying by $\sqrt{2}$ makes the range $[-1, 1]$.
- **Why exponential fog?** Beer–Lambert: each metre removes the same fraction of light.
- **Why is the GPU time 10.9 ms but the frame 16.7 ms?** The display waits for the 60 Hz refresh; the extra 5.8 ms is idle headroom.

- **What did you correct after checking?** IK used the root angle (law of cosines on a, c, b), Cook–Torrance has a visibility term V and a diffuse part, the normal-map slope is divided by cells per tile, and the wind has exact constants.